Break, Continue, dan Pause

Mengatur alur loop di MATLAB

Pemrograman MATLAB

Isi Presentasi

Pengantar

Break

Continue

Pause

Studi kasus: pembagian dengan nol

Kapan dipakai?

Latihan

Ringkasan

Pengantar

Mengingat kembali: cara kerja loop

Sebuah loop mengulang badan loop (baris di antara `for`/`while` dan `end`). Satu kali putaran disebut satu iterasi.

```
for i = 1:4          % kepala loop
    a = i^2;         % \
    disp(a)          % / badan loop
end                  % kembali ke kepala
```

Secara bawaan, setiap iterasi:

1. menjalankan semua baris di badan loop, dari atas ke bawah;
2. kembali ke kepala loop;
3. mengulang sampai nilai `i` habis (`for`) atau kondisi bernilai salah (`while`).

Alur ini kaku: semua iterasi dijalankan penuh, tanpa jeda.

Mengapa alur loop perlu diatur?

Bayangkan Anda memeriksa setumpuk 100 berkas, satu per satu.

`break`
Berkas yang dicari sudah ketemu di tumpukan ke-12. Tidak ada gunanya memeriksa 88 sisanya: **berhenti**.

`continue`
Berkas ke-7 kosong. Tidak perlu dibaca, tetapi pemeriksaan belum selesai: **sisihkan, lanjut ke berkas berikutnya**.

`pause`
Setelah tiap 10 berkas, Anda berhenti sejenak untuk melapor, lalu **melanjutkan dari posisi yang sama**.

Gagasan utama
`break` dan `continue` mengubah alur loop (iterasi mana yang dijalankan). `pause` tidak mengubah alur, hanya mengatur waktu eksekusi.

Tujuan pembelajaran

Setelah materi ini, Anda diharapkan mampu:

1. menjelaskan perbedaan efek `break`, `continue`, dan `pause` terhadap jalannya program;
2. menelusuri (*trace*) loop yang berisi `break`/`continue` dan menebak output-nya;
3. menangani loop bersarang, termasuk keluar dari beberapa loop sekaligus;
4. menghindari jebakan umum, misalnya loop tak berujung akibat `continue` di `while`;
5. memilih alat yang tepat: `break`, `continue`, kondisi `while`, atau operasi vektor.

Break

break: konsep

Definisi

`break` menghentikan loop (`for` atau `while`) seketika. Baris yang tersisa di badan loop tidak dijalankan, iterasi berikutnya tidak pernah terjadi, dan program melompat ke baris pertama setelah `end` milik loop tersebut.

Pola penulisan

```
for ...
    if <kondisi berhenti>
        break;
    end
    <proses>
end
<baris setelah loop>
```

Catatan

- `break` hampir selalu diletakkan di dalam `if`. Tanpa `if`, loop pasti berhenti pada iterasi pertama.
- `break` hanya bermakna di dalam loop. Untuk keluar dari fungsi, gunakan `return`.

break: contoh dasar

Kode

```
for i = 1:10
    if i == 5
        break;
    end
    disp(i)
end
disp('selesai')
```

Output di Command Window

```
1
2
3
4
selesai
```

Perhatikan tiga hal:

- 5 tidak dicetak, karena `break` terjadi sebelum `disp(i)`.
- Nilai 6 sampai `10` tidak pernah diproses.
- `'selesai'` tetap dicetak, karena `break` hanya mengakhiri loop, bukan program.

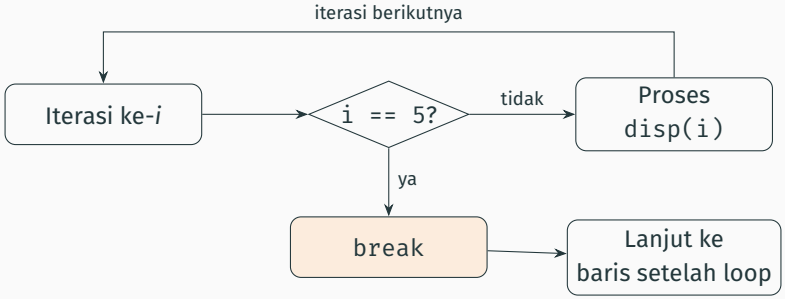
Menelusuri `break` iterasi demi iterasi

`i`	`i == 5?`	Aksi	Output
1	tidak	jalankan `disp(i)`	1
2	tidak	jalankan `disp(i)`	2
3	tidak	jalankan `disp(i)`	3
4	tidak	jalankan `disp(i)`	4
5	ya	`break` → keluar loop	–
6–10	–	tidak pernah dijalankan	–
setelah loop: `disp('selesai')`			`selesai`

Kebiasaan baik

Saat ragu dengan output sebuah loop, buat tabel seperti ini: satu baris per iterasi, catat nilai variabel, hasil kondisi, dan aksi yang terjadi.

Alur eksekusi `break`



Jalur “ya” tidak pernah kembali ke kepala loop. Itulah yang membedakan `break` dari `continue` (lihat bagian berikutnya).

Miskonsepsi: “break keluar dari if”

Kode

```
for i = 1:3
    if i == 2
        disp('masuk if');
        break;
    end
    disp(['akhir iterasi ', num2str(i)]);
end
disp('di luar loop');
```

Output di Command Window

```
akhir iterasi 1
masuk if
di luar loop
```

Yang sering salah dipahami

Walaupun ditulis di dalam `if`, `break` mengakhiri loop yang membungkusnya, bukan `if`-nya. Karena itu `'akhir iterasi 2'` tidak pernah dicetak, dan iterasi `i = 3` tidak terjadi. `if`, `switch`, dan blok lain bukan loop.

break pada while

`break` sering dipasangkan dengan `while true`: loop yang “tak berujung” secara sengaja, dan `break` menjadi satu-satunya pintu keluar.

Kode

```
% Jumlahkan 1+2+3+... sampai total > 20
total = 0;
n = 0;
while true
    n = n + 1;
    total = total + n;
    if total > 20
        break;
    end
end
fprintf('n = %d, total = %d\n', n, total);
```

Output di Command Window

```
n = 6, total = 21
```

Penelusuran `total`: 1, 3, 6, 10, 15, 21. Saat `total` melewati 20, loop berhenti.

Pola ini cocok bila jumlah iterasi tidak diketahui di awal. Pastikan kondisi `break` pasti tercapai; jika tidak, program berjalan selamanya (hentikan dengan `Ctrl+C`).

break pada loop bersarang

Kode

```
for i = 1:3
    for j = 1:3
        if j == 2
            break;    % hanya loop j
        end
        fprintf('i=%d j=%d\n', i, j);
    end
end
```

Output di Command Window

```
i=1 j=1
i=2 j=1
i=3 j=1
```

Aturan

`break` hanya keluar dari loop yang paling dalam tempat ia berada. Di sini, setiap kali `j == 2`, loop `j` berhenti, tetapi loop `i` tetap lanjut ke nilai berikutnya. Akibatnya `j = 3` tidak pernah dicapai, sedangkan `i` tetap berjalan 1, 2, 3.

Keluar dari dua loop sekaligus: pakai penanda (*flag*)

Kode

```
A = [3 7 2; 8 5 1; 4 9 6];
ketemu = false;                 % penanda

for r = 1:size(A,1)
    for c = 1:size(A,2)
        if A(r,c) > 7
            ketemu = true;
            break;              % keluar loop c
        end
    end
    if ketemu
        break;                  % keluar loop r
    end
end

if ketemu
    fprintf('Pertama > 7: baris %d kolom %d\n', ...
        r, c);
end
```

Output di Command Window

```
Pertama > 7: baris 2 kolom 1
```

- `break` pertama hanya keluar dari loop `c`. Penanda `ketemu` "membawa kabar" ke loop `r` agar ikut berhenti.
- Setelah `break`, nilai variabel loop (`r`, `c`) tetap tersimpan, sehingga posisi elemen bisa dipakai di luar loop.
- Di dalam fungsi, `return` juga bisa dipakai untuk keluar dari semua loop sekaligus.

Continue

`continue`: konsep

Definisi

`continue` melewati sisa badan loop pada iterasi saat ini, lalu langsung kembali ke kepala loop untuk memulai iterasi berikutnya. Loop tidak berhenti.

Pola penulisan

```
for ...
    if <kondisi dilewati>
        continue;
    end
    <proses>     % dilewati
end
```

Catatan

- Baris sebelum `continue` tetap dijalankan; hanya baris sesudahnya yang dilewati.
- Pada `for`, variabel loop tetap maju ke nilai berikutnya secara otomatis.
- Seperti `break`, `continue` hanya berlaku pada loop terdalam.

`continue`: contoh dasar

Kode

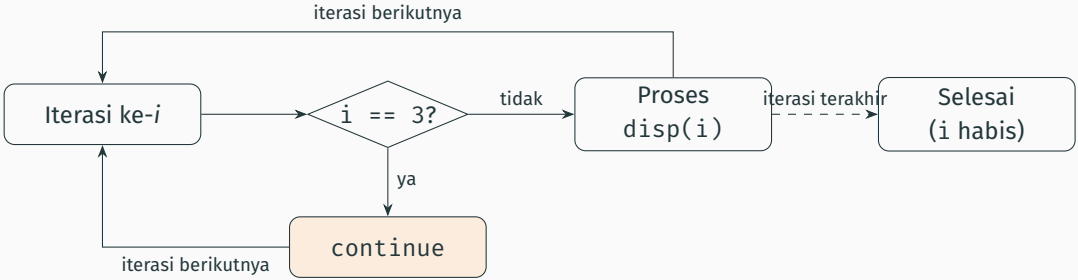
```
for i = 1:5
    if i == 3
        continue;
    end
    disp(i)
end
disp('selesai')
```

Output di Command Window

```
1
2
4
5
selesai
```

`i`	`i == 3?`	Aksi	Output
1, 2	tidak	jalankan `disp(i)`	1, 2
3	ya	`continue` → `disp` dilewati	–
4, 5	tidak	jalankan `disp(i)`	4, 5

Alur eksekusi `continue`



Bandingkan dengan diagram `break`: di sini jalur "ya" kembali ke kepala loop. Loop hanya selesai secara normal, yaitu ketika nilai `i` sudah habis.

Jebakan: `continue` di dalam `while`

Pada `for`, penghitung maju otomatis. Pada `while`, Anda sendiri yang menaikkan penghitung, dan `continue` bisa melompatinya.

Salah: loop tak berujung

```
i = 0;
while i < 5
    if i == 2
        continue;    % i tidak pernah naik!
    end
    disp(i)
    i = i + 1;
end
```

Output `0`, `1`, lalu program "macet": `i` terus bernilai 2, kondisi `i == 2` selalu benar. Hentikan dengan `Ctrl+C`.

Benar: naikkan penghitung lebih dulu

```
i = 0;
while i < 5
    i = i + 1;       % selalu dijalankan
    if i == 3
        continue;
    end
    disp(i)
end
```

Output `1 2 4 5`. Penghitung diletakkan sebelum `continue`, sehingga tidak pernah terlewati.

break vs continue

	`break`	`continue`
Sisa badan loop pada iterasi ini	Dilewati	Dilewati
Iterasi berikutnya	Tidak ada	Tetap dijalankan
Ke mana program melompat	Baris setelah `end`	Kepala loop
Pertanyaan yang dijawab	“Perlu lanjut?”	“Perlu proses yang ini?”
Analogi berkas	Sudah ketemu, berhenti	Berkas kosong, sisihkan

break di `i == 3`, `i = 1:5`

```
1 2 X (4 5 tidak dijalankan)
```

continue di `i == 3`, `i = 1:5`

```
1 2 - 4 5
```

Pause

pause: konsep

Definisi

`pause` menghentikan eksekusi program sementara, lalu melanjutkannya dari baris berikutnya. Tidak ada baris yang dilewati dan tidak ada iterasi yang hilang.

Berbeda dari `break`/`continue`

- Tidak harus berada di dalam loop.
- Tidak mengubah hasil program, hanya kapan baris berikutnya dijalankan.
- Menghapus semua `pause` dari program tidak mengubah output, hanya kecepatannya.

Kegunaan umum

- Memberi waktu pengguna membaca hasil antara.
- Membuat animasi grafik.
- Memberi jeda antarlangkah saat demonstrasi atau *debugging* sederhana.

Bentuk-bentuk pause

Perintah	Efek
`pause(n)`	Jeda *n* detik, lalu lanjut otomatis. *n* boleh pecahan, misalnya `pause(0.5)`.
`pause`	Jeda sampai pengguna menekan tombol apa saja.
`pause(0.05)`	Jeda singkat; MATLAB sempat menggambar ulang grafik, sehingga sering dipakai untuk animasi.
`pause('off')`	Menonaktifkan semua `pause` berikutnya (berguna saat skrip dijalankan tanpa ditunggui).
`pause('on')`	Mengaktifkan kembali `pause`.

Tips

Program yang sedang menunggu pada `pause` (atau loop yang tidak berhenti) dapat dibatalkan dengan `Ctrl+C` di Command Window.

pause(n): hitung mundur

Kode

```
for k = 3:-1:1
    disp(k)
    pause(1);      % tunggu 1 detik
end
disp('Mulai!')
```

Output di Command Window

```
3       (tunggu 1 dtk)
2       (tunggu 1 dtk)
1       (tunggu 1 dtk)
Mulai!
```

- Tanpa `pause`, keempat baris muncul hampir bersamaan.
- Dengan `pause(1)`, output sama persis, tetapi muncul bertahap dalam kurang lebih 3 detik.
- Ini menegaskan bahwa `pause` mengatur waktu, bukan alur.

Contoh pemakaian pause

Animasi sederhana

```
x = linspace(0, 2*pi, 100);
for k = 1:100
    plot(x(1:k), sin(x(1:k)));
    xlim([0 2*pi]); ylim([-1 1]);
    pause(0.05);   % jeda singkat
end
```

Tanpa `pause`, loop selesai sebelum grafik sempat ditampilkan; yang terlihat hanya gambar akhir. Jika hanya perlu memperbarui grafik tanpa menunda, gunakan `drawnow`.

Menunggu pengguna

```
disp('Hasil tahap 1 ...');
disp('Tekan tombol apa saja')
pause;
disp('Lanjut ke tahap 2');
```

Program menunggu tanpa batas waktu. Cocok untuk skrip demonstrasi yang dijalankan bertahap. Saat menjalankan skrip panjang tanpa ditunggui, tambahkan `pause('off')` di awal agar program tidak macet menunggu tombol.

Studi kasus: pembagian dengan nol

Data uji: pembagian dengan 0

Di MATLAB, `x/0` tidak menghasilkan error, melainkan `Inf` (dan `0/0` menghasilkan `NaN`). Program jalan terus, sehingga masalah bisa lolos tanpa disadari.

```
pembilang = [10 20 30 40 50 60];
penyebut  = [ 2  5  0  4  0  3];
```

Data	1	2	3	4	5	6
Hasil	5	4	Inf	10	Inf	20
Jenis	ganjil	genap	bermasalah	genap	bermasalah	genap

Aturan main: (1) pembagi 0 adalah kondisi fatal, proses harus dihentikan; (2) hasil ganjil tidak perlu dicetak, tetapi proses tetap lanjut.

Pertanyaan: aturan mana yang cocok dengan `break`, dan mana yang cocok dengan `continue`?

Versi 1: tanpa pengecekan

```
for i = 1:length(pembilang)
    hasil = pembilang(i) / penyebut(i);
    fprintf('Data ke-%d: %d / %d = %g\n', ...
        i, pembilang(i), penyebut(i), hasil);
end
```

```
Data ke-1: 10 / 2 = 5
Data ke-2: 20 / 5 = 4
Data ke-3: 30 / 0 = Inf
Data ke-4: 40 / 4 = 10
Data ke-5: 50 / 0 = Inf
Data ke-6: 60 / 3 = 20
```

Masalah

Semua dicetak, termasuk `Inf` dan hasil ganjil. Tidak ada tanda bahwa ada yang salah. Jika hasilnya dipakai di `sum` atau rata-rata, hasil akhir ikut menjadi `Inf`.

Versi 2: hanya break

```
for i = 1:length(pembilang)

    if penyebut(i) == 0
        fprintf('Data ke-%d: pembagi 0 -> STOP\n', i);
        break;
    end

    hasil = pembilang(i) / penyebut(i);
    fprintf('Data ke-%d: %d / %d = %d\n', ...
        i, pembilang(i), penyebut(i), hasil);
end
```

```
Data ke-1: 10 / 2 = 5
Data ke-2: 20 / 5 = 4
Data ke-3: pembagi 0 -> STOP
```

Hasil

Pengecekan diletakkan sebelum pembagian, sehingga `Inf` tidak pernah terbentuk. Loop berhenti tepat di data 3; data 4–6 tidak diproses. Namun data ganjil (data 1) masih dicetak, karena aturan kedua belum diterapkan.

Versi 3: break + continue

```
for i = 1:length(pembilang)

    if penyebut(i) == 0
        fprintf('Data ke-%d: pembagi 0 -> STOP\n', i);
        break;        % aturan 1: berhenti total
    end

    hasil = pembilang(i) / penyebut(i);

    if mod(hasil, 2) ~= 0
        continue;     % aturan 2: ganjil, lewati
    end

    fprintf('Data ke-%d: %d / %d = %d\n', ...
        i, pembilang(i), penyebut(i), hasil);
end
```

```
Data ke-2: 20 / 5 = 4
Data ke-3: pembagi 0 -> STOP
```

- Data 1: hasil 5 ganjil → `continue`.
- Data 2: lolos kedua cek → dicetak.
- Data 3: pembagi 0 → `break`.
- Data 4–6: tidak disentuh.

Urutan cek penting: cek fatal (`break`) diletakkan paling atas.

Rangkuman perbandingan

Data	Hasil	Tanpa cek	Hanya `break`	`break+continue`
1	5 (ganjil)	dicetak	dicetak	dilewati
2	4	dicetak	dicetak	dicetak
3	Inf	dicetak	STOP	STOP
4	10	dicetak	tidak diproses	tidak diproses
5	Inf	dicetak	tidak diproses	tidak diproses
6	20	dicetak	tidak diproses	tidak diproses

- `break` mengatur kapan loop berhenti.
- `continue` mengatur iterasi mana yang dilewati, tanpa menghentikan loop.
- Tanpa keduanya, semua berjalan, termasuk `Inf` yang lolos diam-diam.

Kapan dipakai?

Panduan memilih

Pakai `break` jika ...

- Target sudah ditemukan, sisanya tidak perlu dicek
- Hasil sudah konvergen / toleransi tercapai
- Muncul kondisi fatal (data rusak, pembagi 0)
- Input pengguna sudah valid

Pakai `continue` jika ...

- Hanya sebagian data yang tidak perlu diproses
- Ada data kosong / `NaN` yang harus dilewati
- Ingin menghindari `if` bersarang yang dalam
- Menyaring data sebelum perhitungan utama

Pertanyaan penuntun
"Jika kondisi ini terjadi, apakah data sesudahnya masih perlu diproses?" Jika tidak → `break`. Jika ya → `continue`.

Contoh 1: mencari elemen pertama (break)

```
data = [4 8 15 16 23 42];
idx  = 0;                % 0 = belum ketemu

for i = 1:length(data)
    if data(i) > 10
        idx = i;
        break;           % sudah ketemu
    end
end

if idx > 0
    fprintf('Pertama > 10: indeks %d (nilai %d)\n', ...
        idx, data(idx));
else
    disp('Tidak ada elemen > 10');
end
```

```
Pertama > 10: indeks 3 (nilai 15)
```

- Tanpa `break`, loop tetap memeriksa `16, 23, 42`, dan `idx` tertimpa menjadi 6 (elemen terakhir yang > 10, bukan yang pertama).
- Cabang `else` menangani kasus tidak ada yang ditemukan; tanpa itu, `data(0)` akan menghasilkan error.

Contoh 2: iterasi sampai konvergen (break)

Menghitung $\sqrt{2}$ dengan metode Newton: $x_{\text{baru}} = \frac{1}{2}\left(x + \frac{2}{x}\right)$, berhenti saat perubahan sudah sangat kecil.

```
x = 1;
for k = 1:100                % batas aman
    xbaru = 0.5 * (x + 2/x);
    if abs(xbaru - x) < 1e-6
        fprintf('Iterasi %d: x = %.6f\n', ...
            k, xbaru);
        break;               % cukup, hentikan
    end
    x = xbaru;
end
```

```
Iterasi 5: x = 1.414214
```

- Tanpa `break`, 95 iterasi sisanya terbuang percuma.
- Batas `1:100` adalah “pengaman”: jika metode tidak pernah konvergen, loop tetap berhenti. Pola ini umum di metode numerik.

Contoh 3: melewati data kosong (continue)

```
data  = [3 NaN 5 NaN 2];
total = 0;
n     = 0;

for i = 1:length(data)
    if isnan(data(i))
        continue;   % lewati data kosong
    end
    total = total + data(i);
    n     = n + 1;
end

fprintf('Rata-rata = %.2f (%d data valid)\n', ...
    total/n, n);
```

```
Rata-rata = 3.33 (3 data valid)
```

- Tanpa pengecekan, satu `NaN` saja membuat `total` menjadi `NaN`.
- `continue` juga melewati `n = n + 1`, sehingga pembagi rata-rata hanya menghitung data valid.

Contoh 4: validasi input (`while` + `break`)

```
while true
    umur = input('Masukkan umur: ');

    if isscalar(umur) && umur > 0
        break;          % valid, keluar
    end

    disp('Tidak valid, coba lagi.');
end
fprintf('Umur dicatat: %d\n', umur);
```

```
Masukkan umur: -3
Tidak valid, coba lagi.
Masukkan umur: 20
Umur dicatat: 20
```

- Jumlah percobaan tidak diketahui di awal, sehingga `while true` + `break` cocok.
- `isscalar` menolak input kosong (hanya menekan Enter) maupun vektor.

Kapan sebaiknya tidak dipakai

1. Ada cara vektor yang lebih ringkas.
Contoh 1 dan 3 tanpa loop sama sekali:

```
idx  = find(data > 10, 1);        % Contoh 1
rata = mean(data(~isnan(data))); % Contoh 3
```

2. Kondisi berhenti bisa ditulis di `while`.
Contoh 2 tanpa `break`:

```
x = 1;  xbaru = 1.5;  k = 1;
while abs(xbaru - x) >= 1e-6 && k < 100
    x     = xbaru;
    xbaru = 0.5 * (x + 2/x);
    k     = k + 1;
end
```

3. Terlalu banyak `break`/`continue`.
Satu loop dengan banyak titik keluar sulit dilacak. Satu atau dua titik keluar dengan komentar yang jelas biasanya masih wajar.

4. Butuh keluar dari beberapa loop.
`break` hanya memengaruhi loop terdalam. Gunakan penanda (*flag*), atau bungkus loop dalam fungsi dan pakai `return`.

Prinsip
Pilih bentuk yang paling mudah dibaca. `break` dan `continue` adalah alat, bukan kewajiban.

Latihan

Latihan: tebak output-nya

Telusuri setiap kode tanpa menjalankannya, lalu tuliskan output-nya.

Soal 1

```
for k = 1:6
    if mod(k,3) == 0
        continue;
    end
    fprintf('%d ', k);
end
```

Soal 2

```
s = 0;
for k = 1:10
    s = s + k;
    if s >= 15
        break;
    end
end
fprintf('k=%d s=%d', k, s);
```

Soal 3

```
for i = 1:3
    for j = 1:3
        if j == i
            continue;
        end
        fprintf('(%d,%d) ', i, j);
    end
end
```

Soal 4. Diberikan `data = [4 NaN 7 2 -1 5]`. Tulis loop yang menjumlahkan data, dengan aturan: lewati `NaN`, dan hentikan penjumlahan begitu bertemu bilangan negatif. Berapa hasilnya?

Kunci jawaban

Soal 1: `1 2 4 5`

`k = 3` dan `k = 6` habis dibagi 3, sehingga dilewati.

Soal 2: `k=5 s=15`

Penelusuran s: 1, 3, 6, 10, 15. Nilai k saat `break` tetap tersimpan.

Soal 3: `(1,2) (1,3) (2,1) (2,3) (3,1) (3,2)`

Pasangan dengan `i == j` dilewati; `continue` hanya memengaruhi loop `j`.

Soal 4: `total = 13`

```
data  = [4 NaN 7 2 -1 5];
total = 0;
for i = 1:length(data)
    if isnan(data(i))
        continue;      % lewati NaN
    end
    if data(i) < 0
        break;         % berhenti di negatif
    end
    total = total + data(i);
end
disp(total)            % 4 + 7 + 2 = 13
```

Angka 5 di akhir tidak ikut dijumlah karena loop sudah berhenti di `-1`.

Ringkasan

Ringkasan

Perintah	Efek	Lompat ke	Contoh pemakaian
`break`	Keluar dari loop	Baris setelah `end`	Target ketemu, konvergen, data fatal
`continue`	Lewati iterasi ini	Kepala loop	Data `NaN`, data yang disaring
`pause`	Jeda sementara	Baris berikutnya	Animasi, menunggu pengguna

- `break` dan `continue` hanya berlaku di dalam loop, dan hanya pada loop yang paling dalam. Untuk keluar dari beberapa loop, gunakan penanda atau `return`.
- `break` keluar dari loop, bukan dari `if` yang membungkusnya.
- Pada `while`, pastikan penghitung tidak terlewati oleh `continue`.
- `pause` tidak mengubah alur loop, hanya menunda eksekusi.
- Tulis kondisi pengecekan yang jelas agar masalah seperti `Inf` dan `NaN` tidak lolos diam-diam.